

From Nand to Tetris

NPU Extension

Chapter 1: NPU 与系统架构

NPU and System Architecture

Project

Building a Modern Computer From First Principles

Background

Hack 计算机是一台 16 位、64KB RAM 的简单机器：ALU

只有加法和按位运算，没有乘法、没有浮点、没有向量指令。让 Hack CPU 直接执行 CNN 推理在计算量和存储量上都不现实。

真实 SoC 的做法是在 CPU 旁边挂一个专用加速器（NPU）。CPU

只负责“下命令、查状态”，大量重复的乘累加由 NPU 完成。本课程的目标就是在 nand2tetris_verilog 项目里自己搭出这样一个 NPU，并最终跑通一个小分类模型。

本章先不写实现，而是完成系统设计：确定 NPU 的内部结构、Hack 侧的 MMIO 接口、命令/状态寄存器协议，以及 int8 数据格式。

Objective

给出 NPU 的顶层架构图：CPU / Memory / NPU / SRAM 之间的关系。

设计 Hack CPU 与 NPU 之间的内存映射接口（命令寄存器、状态寄存器、数据缓冲区）。

定义 NPU 的命令描述符格式，并估算 8x8 int8 阵列的资源与吞吐量。

Deliverables

- 一份设计文档（Markdown 或 PDF），包含系统框图、数据流、寄存器表。
- NPU 顶层模块 `n2t\_npu\_top` 的端口草案（只声明端口，不需要实现）。
- 一张 MMIO 地址分配表，建议使用 0x7000-0x7FFF 的未占用地址段。

Contract

- 设计文档必须覆盖：命令发起、数据写入、结果回读、完成轮询四个环节。
- 端口草案必须与现有 `n2t\_memory` / `n2t\_computer` 的接线风格一致。
- 所有寄存器必须有明确的位宽、方向（R/W）和复位值。

Resources

- 本仓库 `docs/npu.md`：当前路线图与已搭好的 PE / SystolicArray。
- `solution/05/Computer.v` 与 `solution/05/Memory.v`：Hack 整机接口。
- nand2tetris.org 的 Project 5：Computer Architecture，作为整机背景。

Tips

- 先设计“kick-off + poll”协议：CPU 写命令，NPU 执行，CPU 轮询完成位。
- 把权重加载、图片输入、结果输出都看作“数据搬运”，不要一开始就混进计算细节。

- int8 是本章最重要的数据格式决策：激活和权重都按 8 位有符号整数存放，累加器用 32 位。